```python
# ============================================================
# 1. IMPORT LIBRARIES
# ============================================================

import numpy as np
import pandas as pd


# ============================================================
# 2. LOAD DATASETS
# ============================================================

movies = pd.read_csv('tmdb_5000_movies\\tmdb_5000_movies.csv')
credits = pd.read_csv('tmdb_5000_credits\\tmdb_5000_credits.csv')


# ============================================================
# 3. INITIAL DATASET INSPECTION
# ============================================================

display(movies.shape, credits.shape)  # Display the shapes of both DataFrames

display("movies",movies.head(3))   # Display the first 3 rows of the movies DataFrame
display("credits",credits.head(3)) # Display the first 3 rows of the credits DataFrame

credits.head(2)['cast'].values


# ============================================================
# 4. MERGE DATASETS
# ============================================================

# merging both dataframes
movies = movies.merge(credits,on='title')  # Merge movies and credits DataFrames on the 'title' column
display("Merged DataFrame",movies.head(3)) # Display the first 3 rows of the merged DataFrame
movies.info()


# ============================================================
# 5. SELECT IMPORTANT COLUMNS
# ============================================================

# imp column selection
movies = movies[['movie_id','title','overview','genres','keywords','cast','crew']] # Extract relevant columns
movies.info()
movies.head(2)


# ============================================================
# 6. HANDLE MISSING VALUES
# ============================================================

# Check for null values and handle them
movies.isnull().sum()       # Check for null values
movies.dropna(inplace=True) # Remove rows with null values
movies.isnull().sum()       # Verify that no null values remain


# ============================================================
# 7. HANDLE DUPLICATE VALUES
# ============================================================

# Check for duplicate rows and handle them
movies.duplicated().sum()             # Check for duplicate rows
movies.drop_duplicates(inplace=True)  # Remove duplicate rows
movies.duplicated().sum()             # Verify that no duplicate rows remain


# ============================================================
# 8. ACCESS DATA FROM DATAFRAME
# ============================================================

# Accessing specific data in the DataFrame in drifferent ways
movies.iloc[0,3]      # Access the 'genres' column of the first row if you know the index of the column.
movies.iloc[0].genres # Access the 'genres' column of the first row if you don't know the index of the column.

movies['genres'][0]   # Access the 'genres' column of the first row without using iloc.


# ============================================================
# 9. CONVERT GENRES AND KEYWORDS DATA
# ============================================================

# '[{"id": 28, "name": "Action"}, {"id": 12, "name": "Adventure"}...]'
# extract only name
# and apply on all dictionaries from original list because original list contains less ammount of dictionaries in 'genres' and 'keywords' columns.
# result: ["Action", "Adventure"...]

import ast

def convert(obj):
    L = []
    for i in ast.literal_eval(obj):
        L.append(i['name'])
    return L


movies['genres'] = movies['genres'].apply(convert)      # Convert the 'genres' column to a list of genre names

movies.head(2)                                          # confirm the changes made to the 'genres' column


movies['keywords'] = movies['keywords'].apply(convert)  # Convert the 'keywords' column to a list of keyword names

movies.head(2)                                          # confirm the changes made to the 'keywords' column


# ============================================================
# 10. EXTRACT TOP 3 CAST MEMBERS
# ============================================================

# '[{"id": 28, "name": "Action"}, {"id": 12, "name": "Adventure"}...]'
# extract only name
# and apply on first 3 dictionaries from original list because original list contains huge ammount of dictionaries in 'cast' column.
# result: ["Action", "Adventure"...]


def convert3(obj):
    L = []
    counter = 0
    for i in ast.literal_eval(obj):
        if counter < 3:  # Limit to the first 3 items
            L.append(i['name'])
            counter += 1
        else:
            break
    return L


movies['cast'] = movies['cast'].apply(convert3)          # Convert the 'cast' column to a list of the first 3 cast member names

movies.head(2)                          # confirm the changes made to the 'cast' column


# ============================================================
# 11. EXTRACT DIRECTOR FROM CREW
# ============================================================

# ['[{"credit_id": "52fe48009251416c750aca23", "department": "Directing",...]'
# extract only deparrtment


def fetch_director(obj):
    L = []
    for i in ast.literal_eval(obj):
        if i['job'] == 'Director':
            L.append(i['name'])
            break  # Stop after finding the first director
    return L


movies['crew'] = movies['crew'].apply(fetch_director)     # Convert the 'crew' column to a list containing the director's name

movies.head(2)                          # confirm the changes made to the 'crew' column


# ============================================================
# 12. CONVERT OVERVIEW INTO A LIST OF WORDS
# ============================================================

movies.iloc[0].overview   # Access the 'overview' column of the first row we found it is a string and we will convert it to a list of words.

movies['overview'] = movies['overview'].apply(lambda x:x.split()) # Convert the 'overview' column to a list of words

movies.head(2)                          # confirm the changes made to the 'overview' column


# ============================================================
# 13. REMOVE SPACES FROM TEXT
# ============================================================

movies['genres'] = movies['genres'].apply(lambda x:[i.replace(" ","") for i in x])    # Remove spaces from genre names
movies['keywords'] = movies['keywords'].apply(lambda x:[i.replace(" ","") for i in x])# Remove spaces from keyword names
movies['cast'] = movies['cast'].apply(lambda x:[i.replace(" ","") for i in x])        # Remove spaces from cast names
movies['crew'] = movies['crew'].apply(lambda x:[i.replace(" ","") for i in x])        # Remove spaces from crew names

movies.head(2)                          # confirm the changes made to the 'genres,keywords,cast,crew' column


# ============================================================
# 14. CREATE TAGS COLUMN
# ============================================================

movies['tags'] = movies['overview'] + movies['genres'] + movies['keywords'] + movies['cast'] + movies['crew'] # Combine all relevant columns into a single 'tags' column

movies.head(2)                          # confirm the changes made to the 'tags' column


# ============================================================
# 15. CREATE FINAL DATAFRAME
# ============================================================

new_df = movies[['movie_id','title','tags']]  # Create a new DataFrame with only the 'movie_id', 'title', and 'tags' columns

new_df.head(2)  # Confirm the changes made to the new DataFrame


# ============================================================
# 16. CONVERT TAGS LIST INTO STRING
# ============================================================

new_df['tags'] = new_df['tags'].apply(lambda x:" ".join(x)) # Convert the 'tags' column from a list of words to a single string of words

new_df.head(2)  # Confirm the changes made to the new DataFrame


# ============================================================
# 17. CONVERT TAGS TO LOWERCASE
# ============================================================

new_df['tags'] = new_df['tags'].apply(lambda x:x.lower()) # Convert the 'tags' column to lowercase for uniformity

new_df.head(2)  # Confirm the changes made to the new DataFrame

# ============================================================
# 18. APPLY STEMMING
# ============================================================

# ['loved','love','loves','loving']
# after applying stemming to reduce words to their root form. we will get only 'love' as a feature.
# ['love']

import nltk

from nltk.stem.porter import PorterStemmer
ps = PorterStemmer()


def stem(text):
    y = []
    for i in text.split():
        y.append(ps.stem(i))  # Apply stemming to each word in the text
    return " ".join(y)  # Join the stemmed words back into a single string


new_df['tags'] = new_df['tags'].apply(stem) # Apply the stemming function to the 'tags' column


# ============================================================
# 19. Vactorisation
# ============================================================

from sklearn.feature_extraction.text import CountVectorizer


cv = CountVectorizer(max_features=5000,stop_words='english') # Create a CountVectorizer object to convert text data into numerical features
vector = cv.fit_transform(new_df['tags']).toarray() # Fit and transform the 'tags' column to create a feature matrix


vector # Display the shape of the feature matrix


list(cv.get_feature_names_out()) # Display the feature names


# ============================================================
# 20. COSINE SIMILIARITY
# ============================================================

from sklearn.metrics.pairwise import cosine_similarity


similiarity = cosine_similarity(vector) # Compute the cosine similarity between all pairs of movies based on their feature vectors


# ============================================================
# 21. SORTING THE SIMILIARITY AND RECOMMENDATION FUNCTION
# ============================================================

sorted(list(enumerate(similiarity[0])),reverse=True,key=lambda x:x[1])[1:6] # Sort the movies based on similarity to the first movie and display the top 5 similar movies


def recommend(movie):
    movie_index = new_df[new_df['title'] == movie].index[0] # Get the index of the movie in the DataFrame
    distances = similiarity[movie_index] # Get the similarity scores for the movie
    movies_list = sorted(list(enumerate(distances)),reverse=True,key=lambda x:x[1])[1:6] # Sort the movies based on similarity and get the top 5
    for i in movies_list:
        print(new_df.iloc[i[0]].title) # Print the titles of the recommended movies


# ============================================================
# 22. TEST THE MODEL
# ============================================================

recommend('Batman Begins') # Call the recommend function to get movie recommendations based on 'Batman Begins'
```